

Análise Comparativa do HAProxy e do Nginx

Desempenho e Resiliência como Balanceadores de Carga sob Testes de Estresse e Injeção de Falhas

Universidade Federal de Campina Grande (UFCG) – CEEI / Centro de Informática

Disciplina: Sistemas Distribuídos

Arthur Vieira, Heitor Alves, Isaque Cavalcante, Rafael Alencar

Introdução e Contextualização



O Contexto

Crescimento acelerado de aplicações web exige arquiteturas de alta disponibilidade baseadas em balanceadores de carga.



HAProxy

Proxy focado em alta disponibilidade, suporte avançado a *health checks* ativos e métricas detalhadas de tráfego.



Nginx

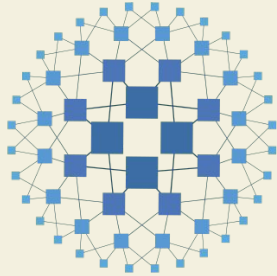
Servidor orientado a eventos (*event-driven*), reconhecido por sua alta eficiência com múltiplas conexões simultâneas.



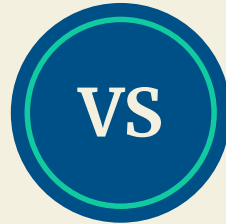
Avaliação

Utilização do *Grafana k6* para simulação real de tráfego sintético, medindo latência detalhada e throughput do sistema.

A Questão da análise



HAPROXY



NGINX



Como o HAProxy e o Nginx diferem em termos de desempenho, eficiência e resiliência na distribuição de tráfego quando submetidos a testes de carga e injeção de falhas?

Objetivos do Estudo



Detecção de Falhas

Avaliar a velocidade de detecção e remoção de nós de backend que sofreram falha através de Health Checks.



Latência e Vazão

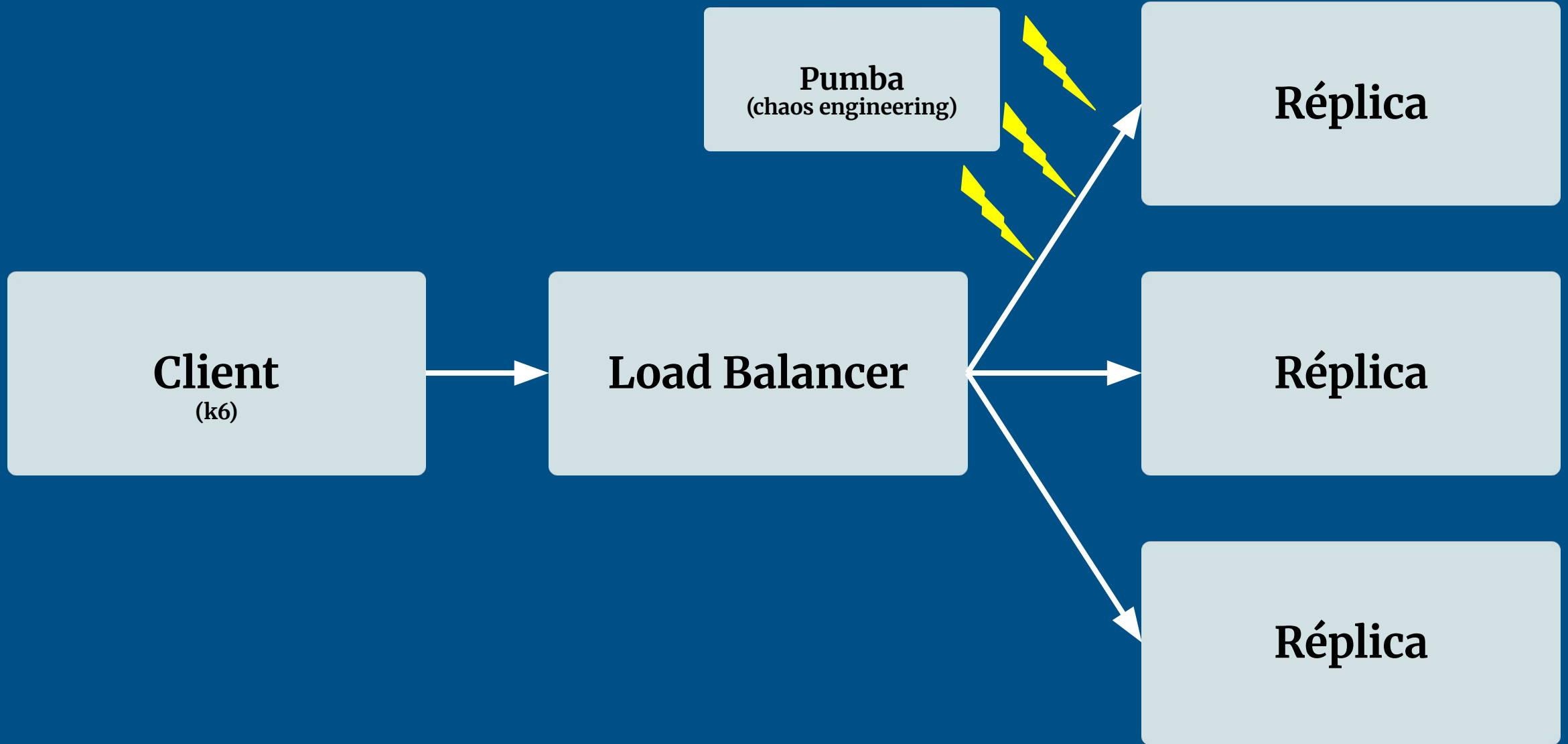
Medir o impacto no tempo de resposta (P95 e P99) e vazão sob picos repentinos de acesso (Flash Crowd).



Recuperação

Analisar a capacidade de recuperação e tolerância a falhas de rede versus falhas de processo.

Arquitetura Experimental



Optimizing Load Balancing Framework for a Distributed Local Network

Validação do Ambiente Restrito:

O artigo prova que implantar um balanceador em uma infraestrutura contêinerizada local já escala o sistema de forma quase linear, dobrando o throughput e reduzindo a carga de CPU pela metade. Como o nosso ambiente de teste utilizou o Docker Compose com limites de recursos em hardware virtualizado, este artigo fundamenta a validade do ambiente experimental.

Decisão Arquitetural

O artigo também comprova que a arquitetura provou-se representativa e totalmente capaz de gerar dados empíricos válidos para decisões, mesmo fora de um grande data center.

Performance And Fault Tolerance Analysis Of Load Balancing Strategies In Microservices Architecture With Controlled Failure Injection

Degradação Catastrófica sob Falhas

(O Gargalo do Round-Robin)

O artigo analisa o Nginx lidando com falhas e prova que, sob o algoritmo Round-Robin estático, a latência de cauda sofre um pico de aumento absurdo (+827%) quando os nós caem, pois o balanceador continua roteando tráfego cegamente para o nó morto até o timeout.

Desempenho sob Condições Normais

Zulman reforça que, com cargas normais, a escolha da ferramenta ou algoritmo quase não afeta o throughput (diferenças menores que **5ms**).

Isso valida **100%** o nosso resultado do Baseline, onde HAProxy e Nginx operaram de forma estatisticamente equivalentes.

Round-robin Algorithm in HAProxy and Nginx Load Balancing Performance Evaluation: a Review

O Impacto do Keep-Alive:

- O artigo destaca que o Nginx só obteve um desempenho superior quando a configuração do Keep-Alive foi explicitamente ativada. Sem ela, o HAProxy demonstrou um desempenho padrão melhor.

Comportamento Padrão e. Otimizado:

- Os autores concluem que, em configurações default, o HAProxy lida melhor com a carga corroborando exatamente com a resiliência superior do HAProxy apontada na conclusão da nossa pesquisa.

Cenários de Teste



Baseline

Carga estável utilizada como grupo de controle.



Spike

Elevação abrupta de VUs (rampas de 5→50 e 100→400 VUs) para testar o esgotamento da fila de conexões.



Failure

Falhas de Rede (delay e loss) e Falhas de Disponibilidade/Processo (kill, stop, pause) nos nós de backend.

Resultados: Cenário Baseline

Aproximadamente

50

Requisições / Segundo

Empate Técnico

Em condições normais de operação, as duas tecnologias mostraram-se perfeitamente intercambiáveis, operando com **0% de erro**.

Latência Média:

HAProxy: 29,29 ms

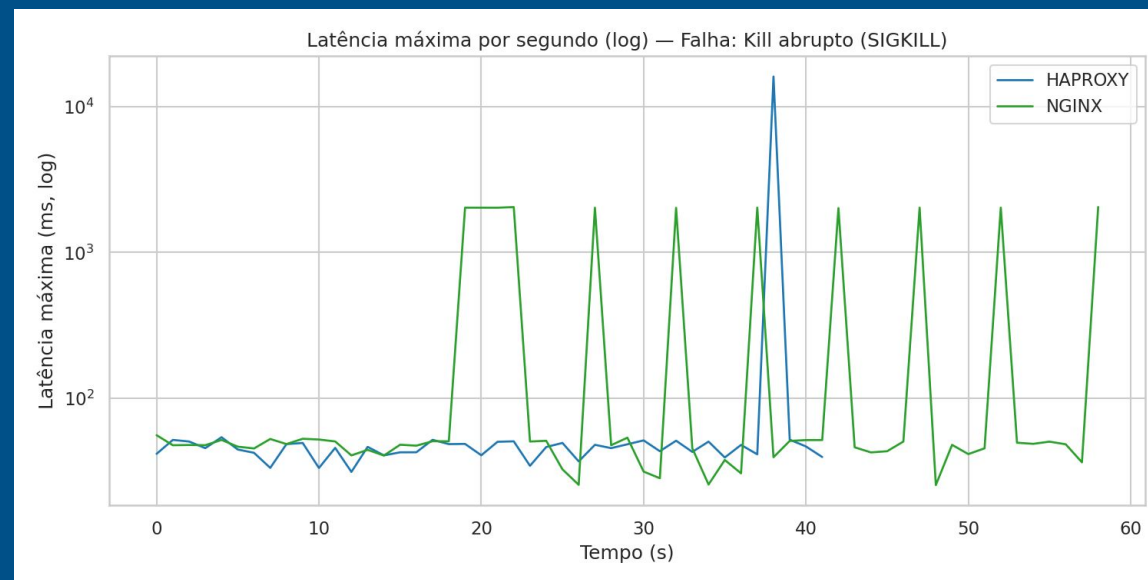
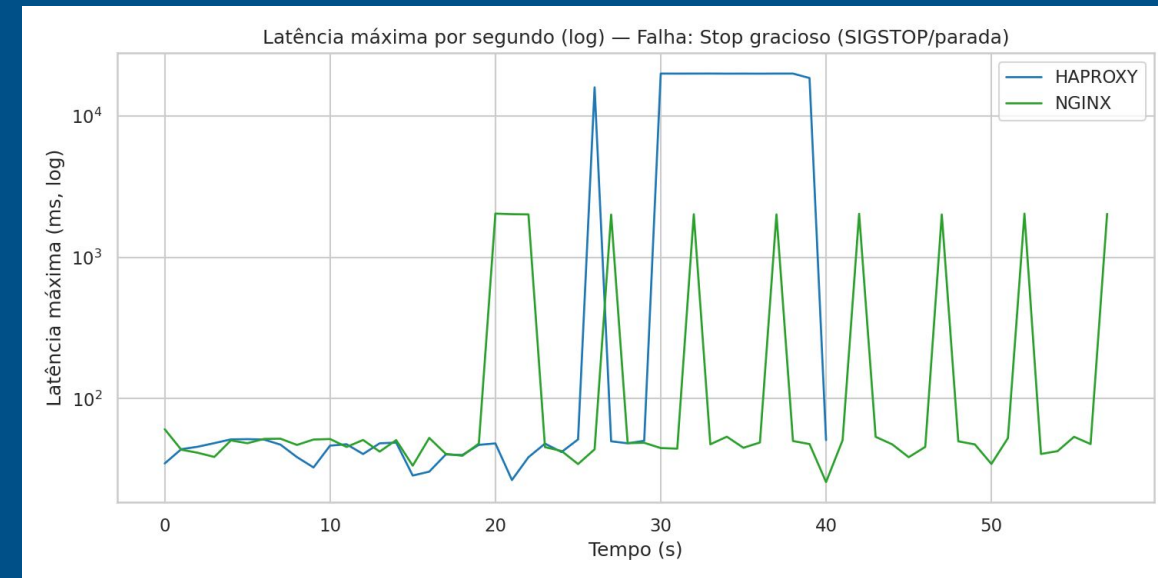
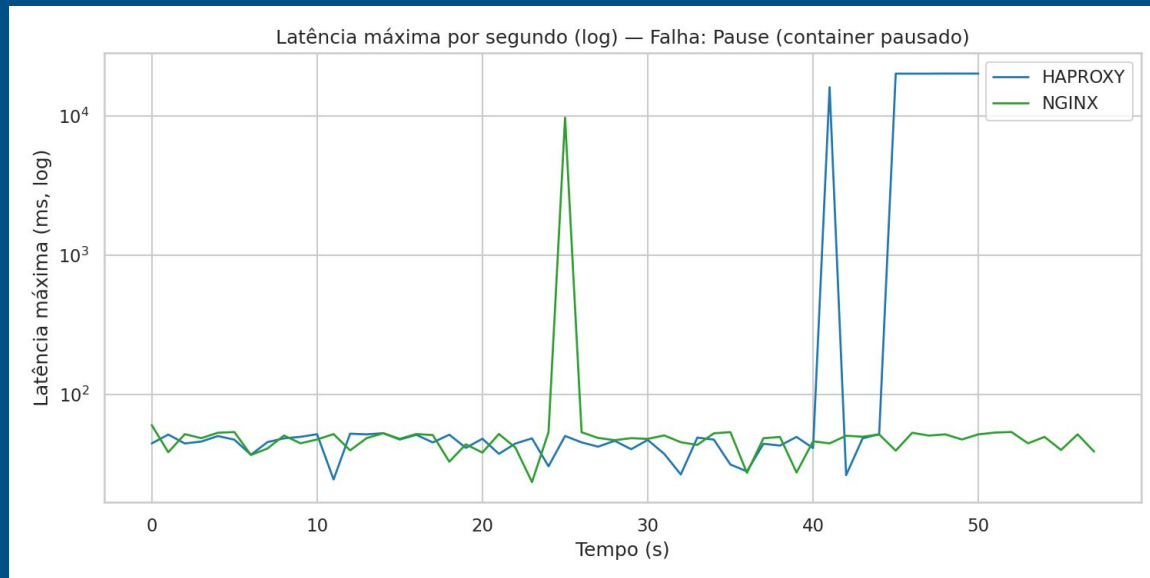
Nginx: 29,41 ms

Latência de Cauda (P99):

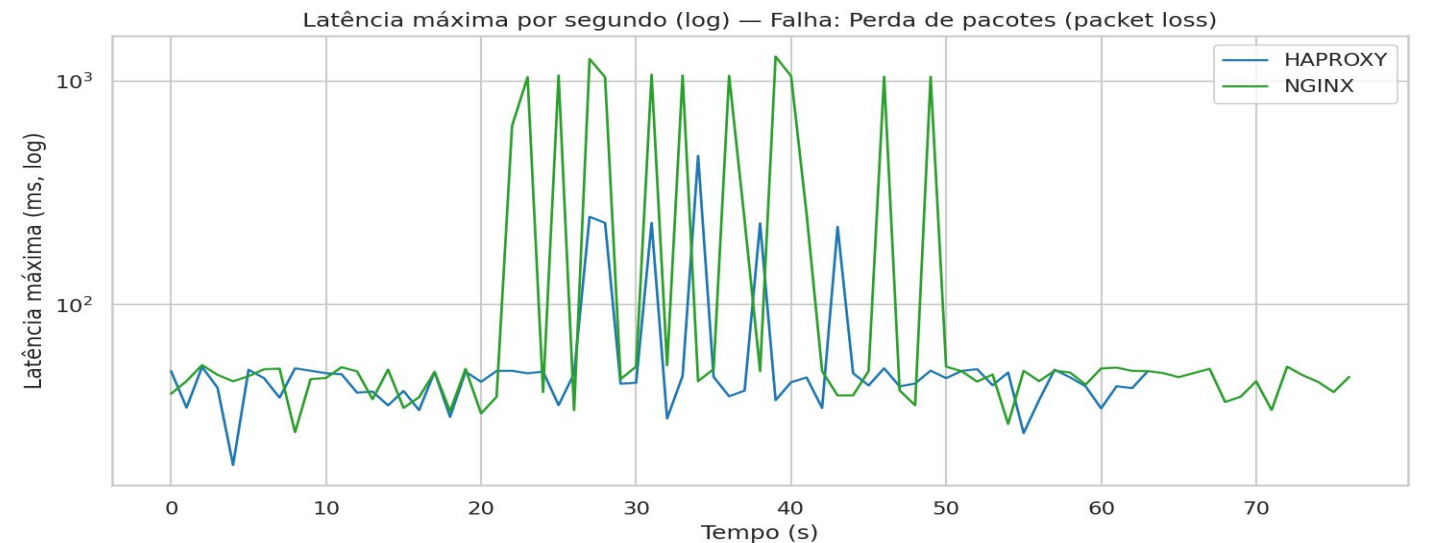
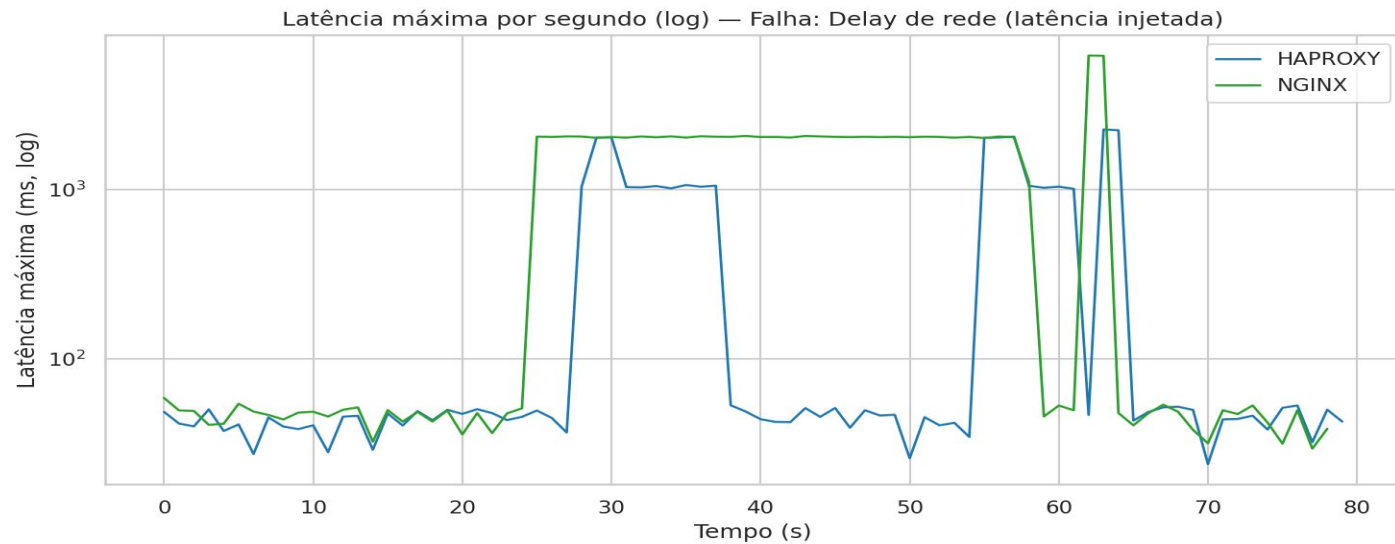
HAProxy: 51,39 ms

Nginx: 51,78 ms

Cenário Failure – réplica indisponível



Cenário Failure – problemas na rede



Falhas de Rede por tipo

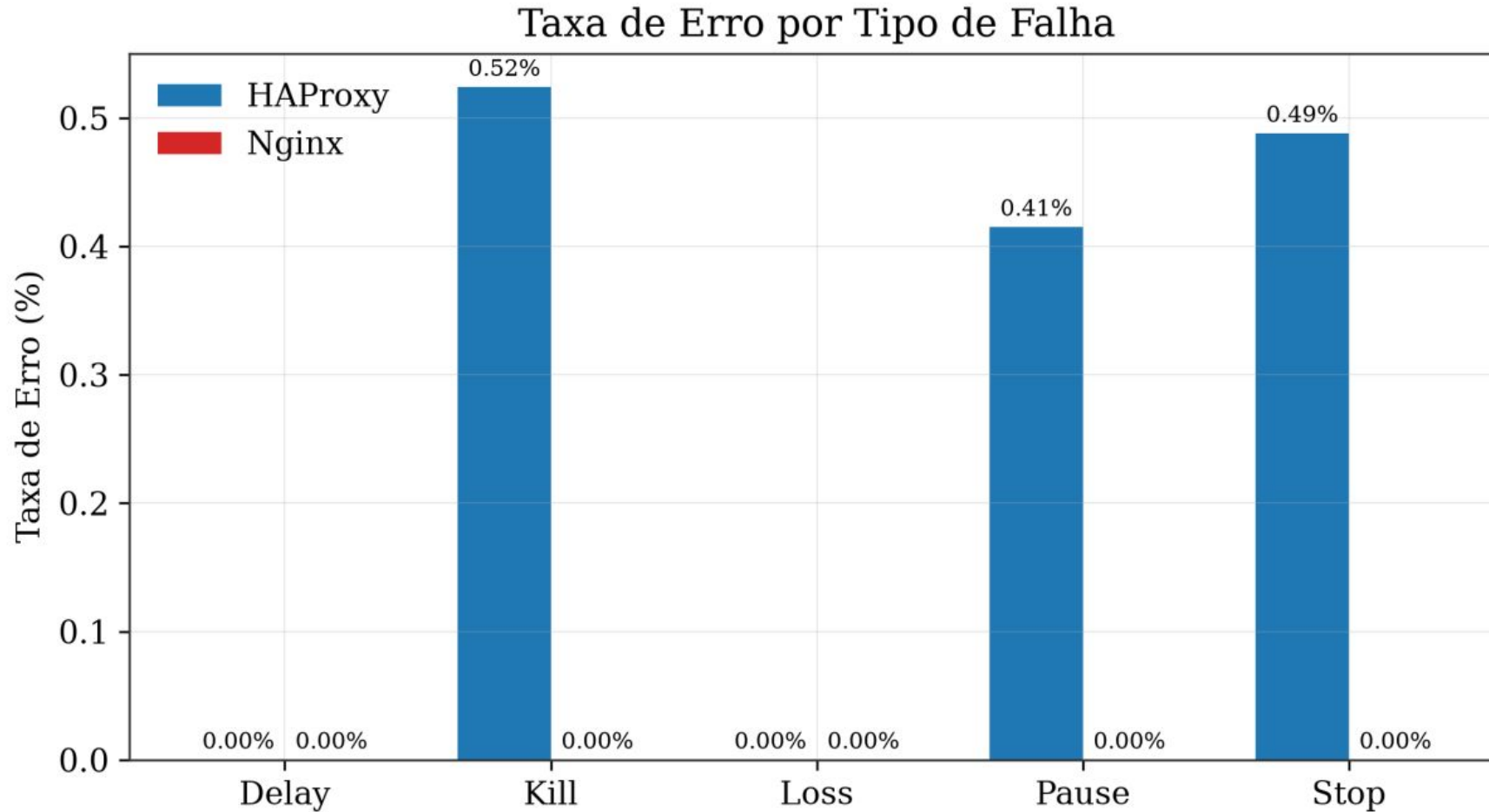
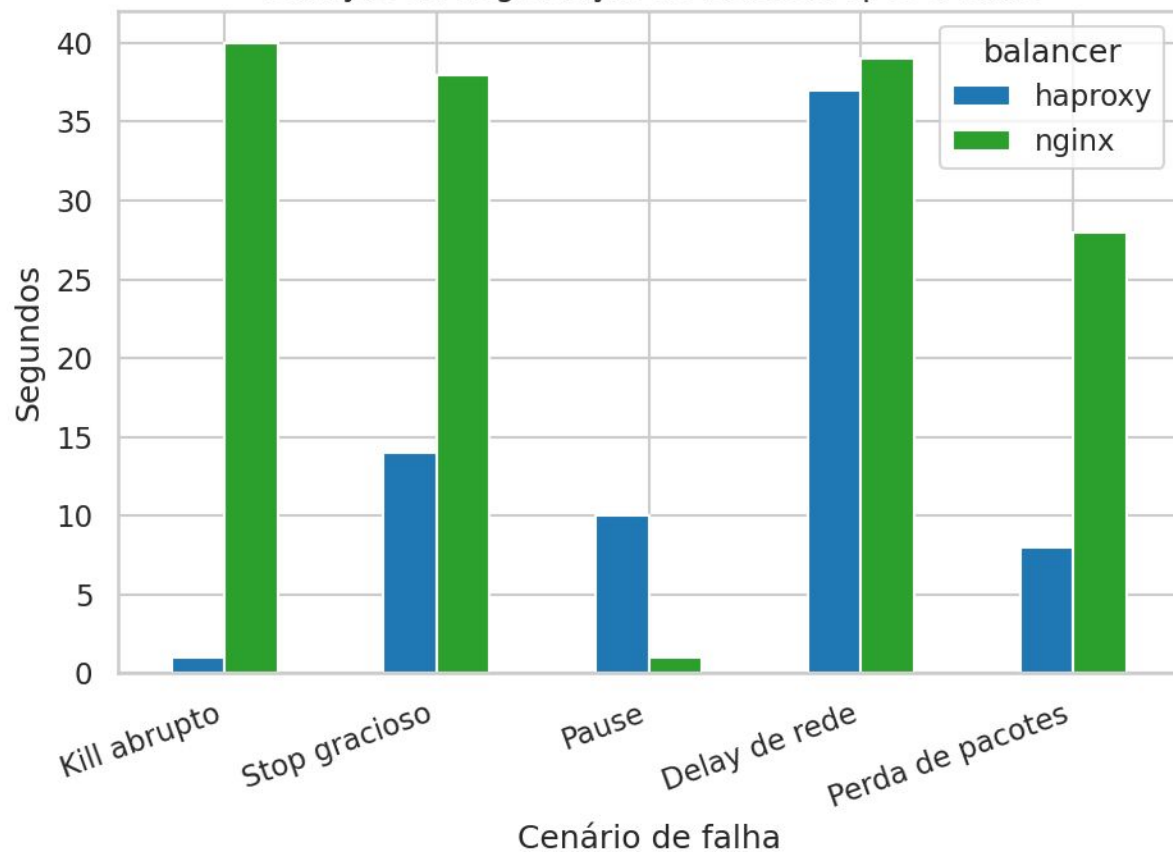


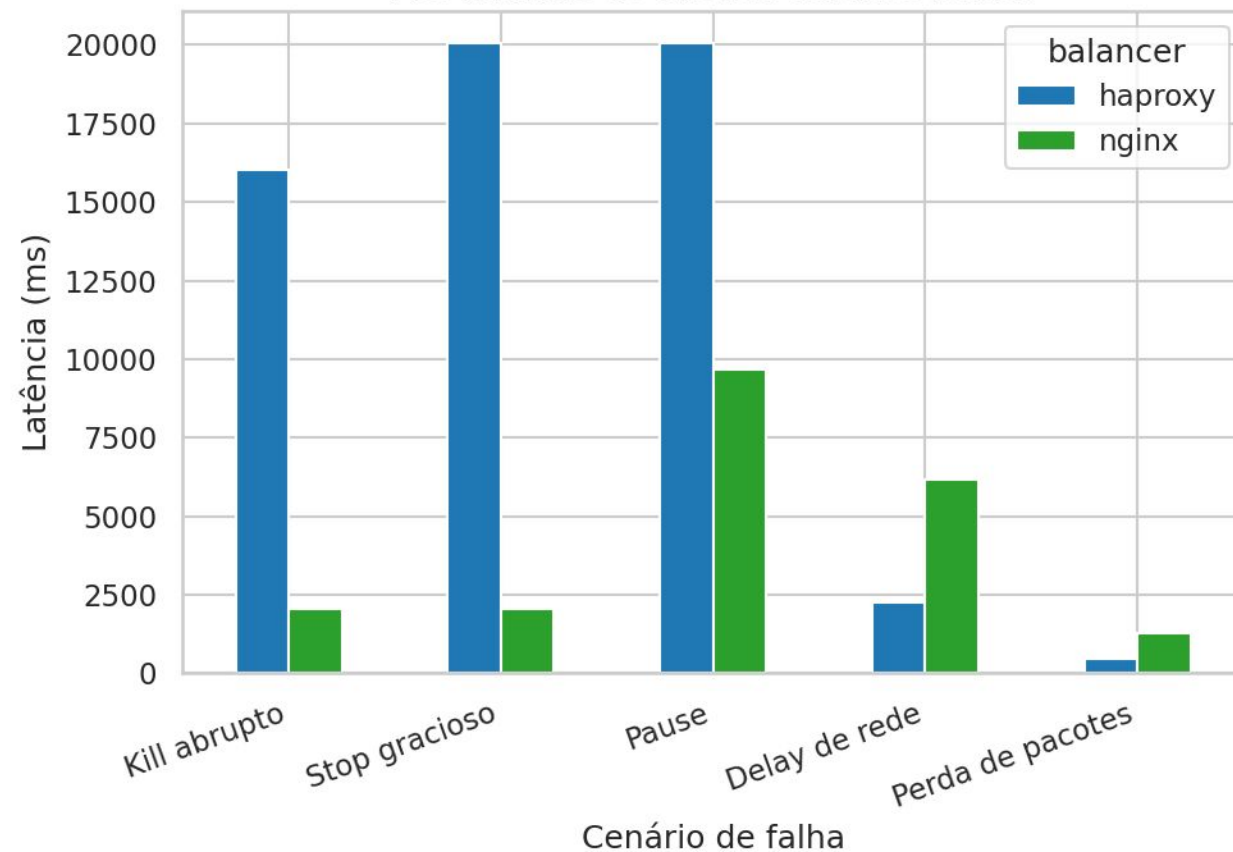
Figura 3 – Taxa de erro por tipo de falha.

Síntese Comparativa

Duração da degradação de latência após a falha



Pico máximo de latência durante a falha



Resultados: Falhas de Rede e Disponibilidade



Falhas de Rede (Delay e Loss): Ambos os balanceadores mantiveram 0% de erro em delay e loss. A perda de pacotes foi absorvida pelas retransmissões TCP, e o delay se refletiu apenas em aumento de latência (pico de ~2,3s no HAProxy e ~6,2s no Nginx para delay; ~0,46s e ~1,3s para loss, respectivamente).



Falhas de Disponibilidade (Kill, Stop, Pause): Nos cenários de disponibilidade, nenhum balanceador é consistentemente melhor: no kill o HAProxy erra um pouco mas resolve rápido, enquanto o Nginx evita erro à custa de retries recorrentes por todo o teste; no pause esse padrão se inverte; e no stop o HAProxy sofre a pior degradação sustentada (14s quase contínuos) contra picos intermitentes do Nginx.



Detecção no HAProxy: O HAProxy levou mais tempo para remover o nó do cluster após a falha do processo. Isso gerou pequenas taxas de erro (0,41% a 0,52%) e janelas ligeiramente maiores de degradação até que o tráfego fosse totalmente redirecionado.

Resiliência contra Falhas de Rede

Cenários Injetados

Atraso de rede (delay) e perda de pacotes (loss) nos contêineres de backend injetados via **Pumba** para testar a robustez do ecossistema.

Desempenho

Ambas as ferramentas analisadas demonstraram excelente resiliência sob estresse:

0%

Taxa de erro registrada pelo HAProxy e Nginx.

Mitigação TCP

Perda de pacotes: Absorvida nativamente pela retransmissão do protocolo TCP.

Atraso (delay): Impacta apenas a latência (Nginx até **344 ms**), sem indisponibilidade da aplicação.

Falhas de Disponibilidade (Queda de Processos)

Cenários Injetados

Simulação de falhas críticas nos processos do backend para avaliar a resiliência:

- **Aborto abrupto (kill):** Parada repentina dos processos.
- **Encerramento (stop):** Finalização controlada.
- **Congelamento (pause):** Suspensão temporária.

O HAProxy

Demonstrou maior sensibilidade e recuperação lenta sob estresse:

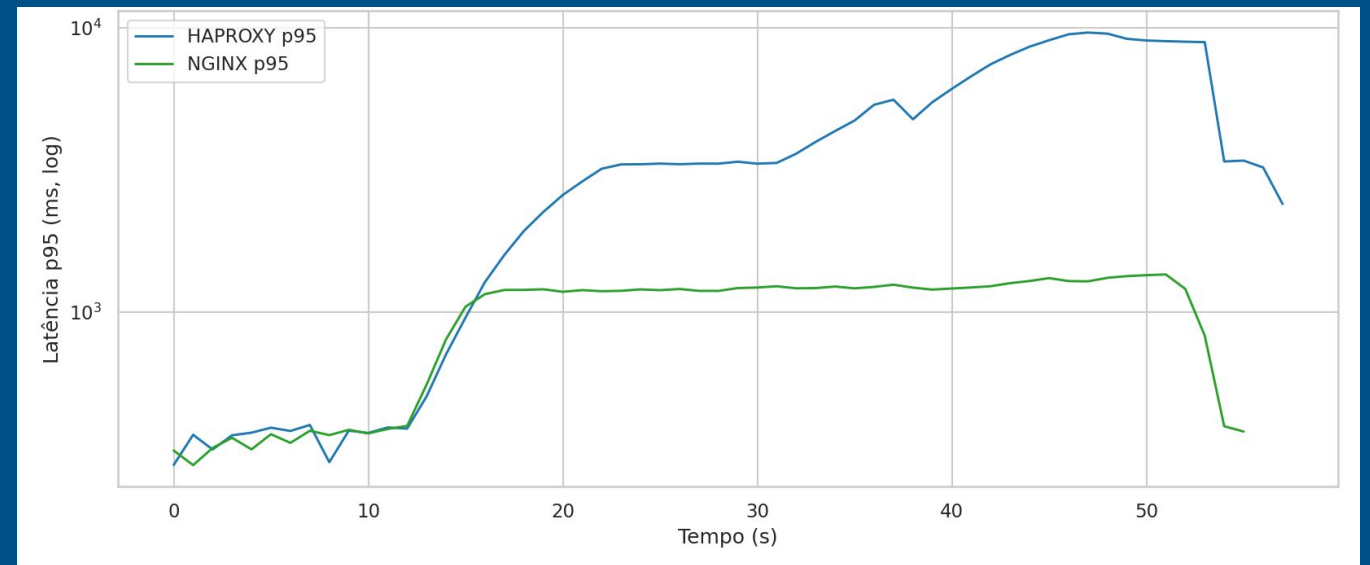
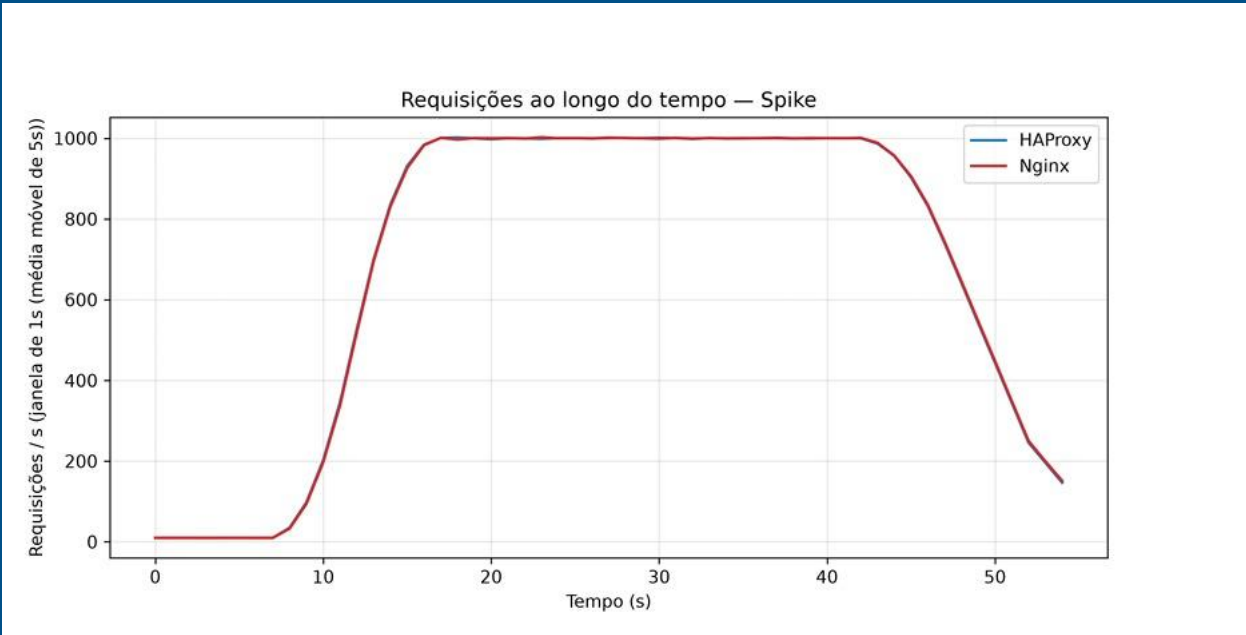
- **Detecção lenta:** Janela de recuperação de ~10s no cenário kill.
- **Pico de latência:** Retenção de requisições gerando até 1.840 ms no cenário stop.
- **Erros:** Pequena taxa de falhas entre 0,41% e 0,52%.

O Nginx

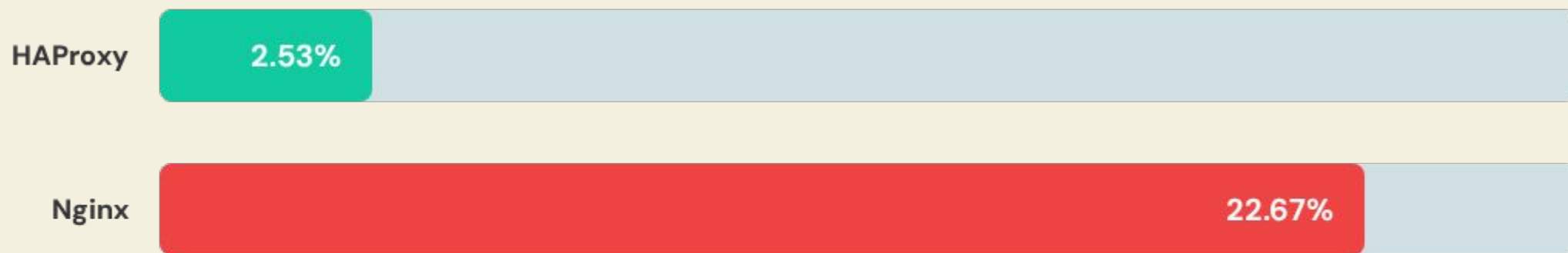
Apresentou excelente desempenho e mitigação imediata de falhas:

- **Estabilidade absoluta:** 0% de erros sob todas as condições testadas.
- **Reação veloz:** Redirecionou o tráfego rapidamente sem colapsar a vazão global.

Cenário Spike: Análise



Síntese Comparativa: Taxa de Erro



A discrepância na taxa de erro ilustra perfeitamente as filosofias arquiteturais distintas: o **HAProxy** prefere atrasar a resposta em vez de negá-la, enquanto o **Nginx** interrompe o processamento para não gargalar o servidor.

Resultados: Cenário Spike

HAProxy: Absorção por Fila

O HAProxy prioriza manter as conexões abertas. Ele absorve a sobrecarga retendo os pacotes, resultando em uma **baixa taxa de erro geral (2,53%)**. No entanto, o custo dessa abordagem é uma latência de cauda significativamente elevada, com o percentil P99 chegando próximo a 4.742 ms.

Nginx: Fail-Fast (Rejeição Rápida)

O Nginx rejeita rapidamente as requisições quando esgota sua capacidade (backlog). Isso gera uma **alta taxa de erro geral (22,67%)**, chegando a 45,55% no teste mais rigoroso. O benefício dessa rejeição é manter a latência sob controle para os pacotes processados (P99 de 679 ms).

Limitações e Trabalhos Futuros

Limitações Observadas

- Execução em ambiente virtualizado com medições de execução única (falta de intervalos de confiança).
- Efeitos de borda causados pelo comportamento do k6 durante retours e timeouts das requisições sintéticas.

Trabalhos Futuros

- Realizar testes modificando parâmetros de Active vs. Passive Health Checks em ambos os proxies.
- Avaliação profunda do impacto no uso de CPU e Memória por nó durante as janelas de degradação.

Lições Aprendidas e Conclusões

Equilíbrio

Não existe "vencedor absoluto"

A escolha tecnológica depende estritamente das regras de negócio e da tolerância a falhas da aplicação.

Resiliência

Aplicações Críticas (Ex: Financeiras)

Beneficiam-se do HAProxy, pois toleram atraso no processamento, mas evitam falhas explícitas ao usuário final.

Velocidade

Alta Responsividade (Ex: Streaming/Games)

Beneficiam-se do Nginx, falhando rápido e permitindo que o cliente gerencie o erro (retries) sem travamentos.

Obrigado!

Principais referências:

AJAZ, Ubaid et al. *Optimizing Load Balancing Framework*, 2026.

PRAMONO, L. et al. *Round-robin Algorithm in HAProxy and Nginx*, 2018.

ZULMAN, M. R. et al. *Performance and Fault Tolerance Analysis*, 2026.